

Lista de implementação de Tratamento de Exceções. Implemente na linguagem Java todos os exercícios propostos. O objetivo é aplicar os conceitos vistos em aula: blocos `try-catch-finally`, exceções `Checked` e `Unchecked`, `throw`, `throws`, e criação de exceções personalizadas.

Formato de Entrega

Instruções para Submissão

A entrega desta lista de exercícios deve ser feita através de um **único arquivo .zip**. Dentro deste arquivo, deve haver uma pasta estruturada para cada exercício (ex: `Exercicio1`, `Exercicio2`, ..., `Exercicio8`).

Atenção: Certifique-se de que cada pasta contenha **todos** os arquivos necessários (`.java`) para a correta compilação e execução do respectivo exercício.

1) Capturando Exceção Nativa Unchecked (`ArithmeticException`)

Escreva um programa que solicite ao usuário dois números inteiros e exiba o resultado da divisão do primeiro pelo segundo.

Requisitos

- Utilize a classe `Scanner` para entrada de dados.
- Utilize um bloco `try-catch` para capturar a exceção `ArithmeticException` caso o usuário insira 0 como divisor.
- Imprima uma mensagem amigável alertando sobre a impossibilidade da divisão por zero, em vez de deixar o programa fechar abruptamente.

2) Tratamento de Múltiplas Exceções (Ordem dos blocos)

Expanda o programa do exercício anterior para lidar também com erros de entrada de dados.

Requisitos

- Adicione um novo bloco `catch` para capturar a `InputMismatchException` (lançada quando o usuário digita letras ou símbolos onde se espera um inteiro).
- Lembre-se de importar o pacote `java.util.InputMismatchException`.
- Demonstre a correta estruturação e ordem adequada dos múltiplos blocos `catch`.

3) Uso do Multi-Catch (Sintaxe Java 7+)

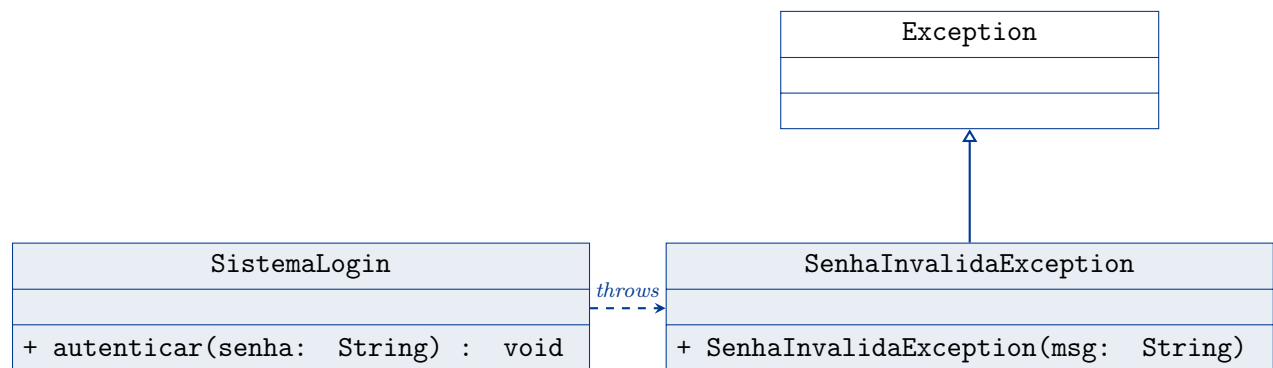
Modifique o exercício anterior unindo os blocos de captura da `ArithmeticException` e `InputMismatchException` em um único bloco.

Requisitos

- Utilize o operador de Multi-Catch (1) para capturar ambas as exceções no mesmo bloco `catch`.
- Imprima uma mensagem genérica como: *"Erro de entrada de dados ou operação matemática inválida"*.

4) Exceção Personalizada Verificada (Checked)

Crie um sistema simples de login que valida o tamanho da senha.



Requisitos

- Crie uma classe de exceção personalizada chamada `SenhaInvalidaException` que herde diretamente de `Exception`.
- Crie uma classe `SistemaLogin` com um método `autenticar(String senha)` que lance ativamente (`throw`) essa exceção caso a senha possua menos de 6 caracteres.
- Teste o método na classe `Main`, encapsulando a chamada em um bloco `try-catch` e imprimindo a mensagem do erro utilizando o método `getMessage()`.

5) Lançamento Ativo de Exceção Nativa (`IllegalArgumentException`)

Modele uma classe `Pessoa` que não permite idades negativas.

Pessoa
- idade : int
+ getIdade() : int
+ setIdade(idade: int) : void

Requisitos

- Crie a classe `Pessoa` com o atributo privado `idade` e seu método `setIdade(int idade)`.
- Dentro do método setter, verifique se a idade passada é negativa.
- Se for negativa, lance ativamente (`throw`) uma `IllegalArgumentException` (exceção nativa não verificada) com a mensagem *"A idade não pode ser negativa"*.
- Realize a captura desta exceção na classe principal (`Main`).

6) Garantia de Execução com bloco Finally

Simule o uso de um recurso que necessita ser finalizado de forma garantida após o uso, independentemente de erros.

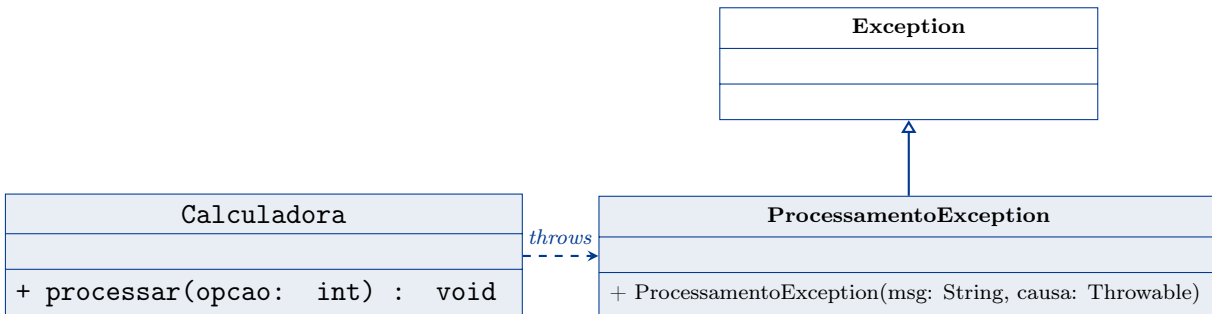
ConexaoBanco
+ conectar() : void
+ desconectar() : void

Requisitos

- Crie uma classe `ConexaoBanco` com os métodos `conectar()` (que imprime "Conectado") e `desconectar()` (que imprime "Desconectado").
- No método `Main`, chame `conectar()` dentro de um bloco `try`.
- Simule um erro no meio do processamento (ex: lançando uma exceção qualquer ou forçando uma divisão por zero).
- Obrigatoriamente, dentro de um bloco `finally`, chame o método `desconectar()` garantindo a "liberação do recurso", comprovando no console que ele executou.

7) Encadeamento de Exceções (Chained Exceptions)

Crie um mecanismo de abstração de falhas capturando exceções nativas e preservando a causa original.

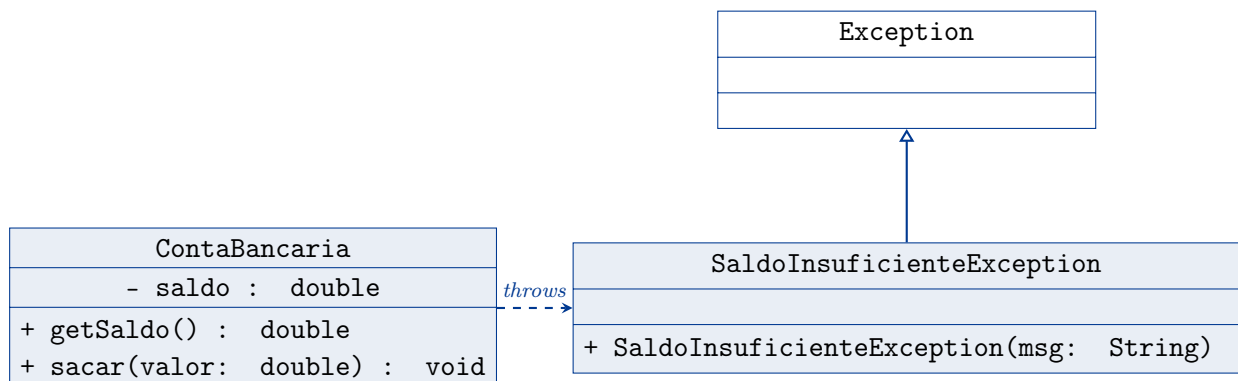


Requisitos

- Crie uma classe `ProcessamentoException` (Checked) que tenha um construtor recebendo uma `String` e um objeto `Throwable` (a causa raiz). Lembre-se de repassar para a superclasse utilizando `super(mensagem, causa)`.
- Crie uma classe `Calculadora` com o método `processar(int opcao)` que simule diferentes falhas: se `opcao == 1`, provoque uma `ArithmeticException` (divisão por zero); se `opcao == 2`, provoque uma `NullPointerException` (acesso a objeto nulo).
- O método `processar()` deve capturar essas exceções (utilizando múltiplos blocos `catch` ou um `Multi-catch`) e lançar a `ProcessamentoException` informando a causa original (o objeto da exceção nativa).
- No `Main`, teste as duas opções e trate o erro imprimindo o *stack trace* para verificar se a *"Caused by:"* foi preservada na saída, mostrando corretamente se foi `ArithmeticException` ou `NullPointerException`.

8) Cenário de Domínio: Conta Bancária

Implemente um sistema completo de transação bancária aplicando as regras de POO e tratamento de erros.



Requisitos

- Crie a exceção customizada `SaldoInsuficienteException`.
- Crie a classe `ContaBancaria` com o atributo `saldo`.
- Crie o método `sacar(double valor)`. Se não houver saldo suficiente, lance a `SaldoInsuficienteException`. Se o valor de saque for negativo ou zero, lance uma `IllegalArgumentException`.
- No `Main`, instancie a conta e teste os fluxos: sacar com sucesso, tentar sacar sem saldo, e tentar sacar um valor negativo.
- Trate adequadamente todas as exceções utilizando os respectivos blocos `catch`.